TraceSentry: Provenance-Aware Oversight of Multi-Agent Attacks Through Stigmergic Coordination Detection[1]

Pranesh Nur
1
Independent
Researcher

With
Apart Research

Abstract

Multi-agent attacks can be difficult to detect because malicious coordination may be distributed across agents whose individual actions appear benign. We present TraceSentry, a provenance-aware oversight system developed for the Swarm Traces Track 4 challenge. TraceSentry first reconstructs multi-agent activity as a tripartite provenance graph linking execution events, shared infrastructure, and attack capabilities. This reconstruction exposes coordination patterns that are difficult to identify through payload inspection alone, including stigmergic propagation through shared credentials and artifacts without direct agent-to-agent communication. Building on these findings, TraceSentry uses a two-stage detection pipeline: a low-latency, stateful security gateway filters benign background activity using observable egress metadata, while a classical machine-learning stage analyzes graph-derived features from escalated events to distinguish active attackers, downstream propagators, and benign false alarms.On the evaluated Swarm Traces incident, the system reconstructed the attack lineage, identified the earliest actionable origin, and quantified downstream propagation. In EGRESS_ONLY mode, the Stage 1 gateway reduced the volume of events requiring deeper inspection while retaining substantial detection capability. The Stage 2 TraceSentry ensemble achieved 0.9831 precision, 0.9733 F1-score, and 6.99 μs latency per sample on the holdout evaluation. These results suggest that provenance-based structural signals can turn hidden multi-agent coordination into actionable oversight while maintaining the efficiency required for high-volume swarm monitoring.

1. Introduction

Autonomous multi-agent systems introduce a distinct oversight problem: an attack may be distributed across many agents whose individual actions appear legitimate when inspected in isolation. In a coordinated swarm, a single compromised event can propagate through shared credentials, files, services, or other environmental state,

1 Research conducted at the Swarm Alignment Hackathon, November 2026

allowing later agents to participate in the attack without producing an obviously malicious payload. This creates a gap between **event-level inspection** and **swarm-level behavior**. Detecting whether an individual action is suspicious is not necessarily sufficient to determine whether that action is part of a coordinated attack.

This problem is illustrated by the Swarm Traces incident used in the Track 4 challenge. The task is to reconstruct how information spread through the swarm, determine whether coordination could have been identified early, and document what an investigator would still miss. The trace data is particularly challenging because agent identities are redacted and coordination does not necessarily occur through direct agent-to-agent communication. Instead, agents may interact indirectly by reading or modifying shared environmental artifacts. Such behavior is consistent with **stigmergic coordination**, where environmental state acts as the medium through which agents influence subsequent behavior. In this setting, an investigator who examines only message content or isolated execution events may miss the relationships connecting otherwise benign-looking actions.

We introduce **TraceSentry**, a provenance-aware oversight system designed to address this gap. The system first reconstructs the incident as a heterogeneous provenance graph linking execution events with shared infrastructure and attack capabilities. Rather than treating every shared resource as equally informative, the graph weights relationships according to the rarity and operational significance of the shared artifact or capability. This makes it possible to reconstruct coordination episodes, identify the earliest actionable origin in the observed lineage, and trace how compromised resources propagate through the swarm.

The resulting forensic structure also motivates a practical detection architecture. Because the vast majority of events in a swarm may be benign, applying expensive analysis uniformly to the entire event stream is inefficient. TraceSentry therefore uses a two-stage pipeline. A stateful security gateway acts as a fast-path filter, using observable event and egress metadata to eliminate benign activity and escalate suspicious events. A second-stage classical machine-learning model then evaluates graph-derived structural features from the smaller suspect population. This separates inexpensive broad screening from more detailed swarm-level reasoning.

This paper makes three contributions:

1. **Provenance-based swarm forensics.** We construct a tripartite provenance representation that exposes coordination through shared infrastructure and capabilities, including stigmergic propagation that is not apparent from direct communication or isolated payload inspection.

2. **A two-stage practical oversight pipeline.** We combine stateful security policy enforcement with classical machine learning over graph-derived features, allowing high-volume event streams to be screened efficiently while reserving more expensive analysis for suspicious activity.

3. **Empirical evaluation of early detection and its blind spots.** On the evaluated Swarm Traces incident, we test forensic reconstruction, early gateway detection, second-stage classification, and downstream containment. We also explicitly analyze cases that remain invisible to the architecture, including coordination through semantic channels and adaptive behavior outside the monitored metadata.

2. Related Work

Recent work on multi-agent systems has shown that autonomous agents can coordinate through shared environmental state rather than explicit communication. Buehler et al. (2026) describe stigmergic coordination in populations of language-model agents, where persistent files and databases become mechanisms for influencing subsequent agents. This challenges monitoring approaches centered on direct agent-to-agent communication, because coordinated behavior may emerge from interactions with shared artifacts rather than visible dialogue. Wijk et al. (2026) further develop this perspective in their analysis of coordinated agent intrusions, introducing concepts such as coordination episodes, prospective episode discovery, first-contact signals, and inherited state. These ideas motivate treating distributed activity across agent lifecycles as a single investigative object rather than analyzing each execution independently. TraceSentry builds on this line of work by operationalizing these ideas as a provenance graph and using the resulting structure for both forensic reconstruction and early oversight.

A second body of work concerns causal provenance and intrusion attribution. The Diamond Model of Intrusion Analysis (Caltagirone et al., 2013) represents attacks through relationships among adversaries, capabilities, infrastructure, and victims, and provides a basis for pivoting across shared infrastructure when adversary identity is unavailable. This is particularly relevant to redacted swarm traces, where agent identities cannot serve as the primary linkage mechanism. HOLMES (Milajerdi et al., 2019) similarly uses causal

provenance graphs and dynamic taint tracking to correlate low-level events into higher-level attack scenarios. TraceSentry extends these provenance ideas to multi-agent swarm telemetry by explicitly representing execution events, shared infrastructure, and capabilities as a tripartite graph, then weighting shared artifacts according to their evidentiary value. This allows common infrastructure to contribute less coordination signal than rare or suspicious shared state.

A third line of work concerns practical oversight under computational constraints. Greenblatt et al. (2024) demonstrate a control strategy in which a fast monitor screens actions and defers suspicious cases to a more expensive auditor. TraceSentry adopts this general cascaded-monitoring principle, but applies it to a different failure mode: the relevant signal is not only the content of an individual action, but its relationship to previous events and shared environmental state. Existing network-monitoring architectures such as Zeek likewise separate fast event processing from stateful policy evaluation (Paxson, 1999), providing a systems precedent for TraceSentry's low-latency gateway stage. In contrast, white-box approaches such as Rose et al. (2026) investigate multi-agent collusion through internal model representations; these methods require access to model internals that is unavailable in a black-box forensic setting such as Swarm Traces.

Finally, TraceSentry uses classical tabular machine learning rather than a neural or language-model monitor for its second-stage classifier. Grinsztajn et al. (2022) find that tree-based methods remain strong on heterogeneous tabular data, motivating the use of Random Forest and XGBoost over an additional neural model for graph-derived features. This choice also supports the operational requirement for low-latency, interpretable inference.

The gap addressed by TraceSentry is therefore not the absence of individual techniques for swarm analysis, provenance, security monitoring, or tabular classification. Rather, existing work leaves open how these ideas can be combined into an **end-to-end, black-box oversight pipeline** that reconstructs hidden multi-agent coordination and then converts those structural signals into an actionable early-warning mechanism. TraceSentry addresses this gap by connecting provenance-based forensic reconstruction to a two-stage gateway and machine-learning architecture, while explicitly evaluating what remains undetectable under the available observations.

3. Methods

3.1 Data Ingestion and Event Normalization

TraceSentry operates on the redacted Swarm Traces JSONL corpus. The raw records are validated with a typed schema and streamed line-by-line rather than loaded as an unstructured corpus. Each record is normalized into a common event representation containing an event identifier, sequence position, timestamp, sender/receiver information when available, action type, and extracted attack artifacts. The parser extracts task identifiers, destinations, file paths, command-injection indicators, and other structured fields from the raw payload text. The normalized representation is used by the forensic components, while the gateway operates directly on the validated raw record and shares the same deterministic artifact and capability extractors.

The implementation uses deterministic regular-expression extraction for infrastructure and capability indicators. Infrastructure entities include credentials, checksums, task identifiers, destinations, and known shared C2 or dead-drop boards. Capability indicators include remote command execution (RCE), covert exfiltration, defense evasion, and synchronization probes. These extractors provide a common interface for the provenance graph and the online checker rather than maintaining separate feature definitions for each component.

3.2 Tripartite Provenance Graph

We represent the incident as a directed heterogeneous graph

$G=(V,E)$

with three node classes: **payload events**, **infrastructure artifacts**, and **attack capabilities**. A payload node represents one trace event. Infrastructure nodes represent shared resources such as credentials, checksums, task identifiers, destinations, and coordination boards. Capability nodes represent detected behavioral capabilities such as RCE or covert exfiltration.

Three principal edge relations are constructed. First, explicit parent-child relationships in the trace become **lineage edges**. Second, a payload is connected to every infrastructure artifact it uses. Third, a payload is connected

to the capabilities detected in its contents. Infrastructure and capability nodes also retain reverse links to their consuming payloads, allowing shared artifacts to be used as pivots between otherwise disconnected events.

To suppress common-resource noise, each shared artifact is assigned a Newman-style inverse-frequency weight,

$$w_k = \begin{cases} \dfrac{1}{d_k - 1} & d_k > 1 \\ 0 & d_k \leq 1 \end{cases}$$

where d_k is the number of unique payloads connected to artifact k. This weight is multiplied by an artifact- or capability-specific importance factor to obtain a weighted coordination score. Rare shared credentials or coordination artifacts therefore contribute more evidence of linkage than ubiquitous destinations. Activity threads for each artifact are ordered by depth and sequence index, and the first payload in the thread is recorded as the artifact's first observed contact. The graph implementation avoids materializing a full payload-to-payload projection; two-hop relationships are recovered on demand through payload → artifact → payload traversal.

3.3 Patient-Zero Attribution and Forensic Reconstruction

For forensic attribution, TraceSentry examines all depth-zero payloads as candidate origins. Each candidate is scored using three signals: **artifact novelty**, **capability pairing**, and **downstream reach**. Artifact novelty measures whether the root event is the first observed payload to introduce credentials, coordination artifacts, or other shared infrastructure. Capability pairing measures the severity-weighted attack capabilities associated with the root. Downstream reach combines ordinary lineage descendants with stigmergic consumers that later interacted with artifacts first introduced by the candidate.

The resulting composite score rewards candidates that simultaneously introduce novel shared state, exhibit attack capability, and reach a large downstream population. Candidates are classified into operational tiers ranging from weaponized originators to stigmergic asset seeders, while roots without relevant capabilities or originated shared assets are discarded. The resulting ranking is used to identify the earliest actionable origin and to reconstruct the propagation structure of the incident.

For visualization, the implementation renders a two-hop neighborhood around a selected root payload, showing payload, infrastructure, and capability nodes separately. This local view is used as a human-readable representation of the provenance relationships recovered by the graph.

3.4 Stage 1: Stateful Gateway Early Checker

The online oversight component is implemented as a stateful gateway that processes events sequentially. It maintains three evolving structures: an approved task manifest, a taint registry for compromised credentials and code checksums, and an exposure counter recording repeated use of previously unseen assets.

For every event, three classes of evidence are accumulated: infrastructure violations, shared-code/checksum provenance, and capability signatures. Infrastructure evidence includes undeclared or previously tainted credentials, repeated credential reuse, and known C2/dead-drop markers. In full packet-inspection mode, additional evidence is obtained from checksum reuse and exploit signatures such as RCE, covert exfiltration, defense evasion, and synchronization probes.

The component operates with two thresholds. Events whose composite suspicion score reaches the escalation threshold are passed to Stage 2. A higher threshold commits newly observed credentials or checksums to the taint registry. Taint inheritance is explicitly temporal: an asset contributes inherited-taint evidence only when it was committed at an earlier sequence position. State updates occur only after the current event has been scored, preventing the current event from influencing its own historical evidence.

We evaluate two modes as an ablation. **FULL_DPI** enables checksum and payload-capability inspection. **EGRESS_ONLY** disables those body-derived checks while retaining infrastructure-level metadata checks. Thus the comparison tests how much detection and filtering capability can be retained when message contents are not inspected.

3.5 Stage 2: Provenance-Derived Feature Classification

Only events escalated by Stage 1 enter the second-stage dataset. Stage-2 labels were generated from confirmed incident markers in the trace corpus using the same provenance-resolution procedure used to establish the attack set. Each escalated event is represented by eight numerical features:

$$X=\left[s_{\text{early}}, w_{\text{Newman}}, d_k, g_{\text{BM25}}, r_{\text{NIST}}, d_{\text{causal}}, ell_{\text{payload}}, I_{\text{stigm}}\right]$$

Here, s_{early} is the Stage 1 suspicion score; $w_{\text{Newman}}$and d_k capture artifact rarity and reuse; g_{BM25} is a saturating artifact-sharing measure,

$$g_{\text{BM25}}=\alpha_k \frac{d_k}{d_k+20}$$

d_{causal} is trace depth; ell_{payload} is payload length; and I_{stigm} indicates an event that accesses shared credential/checksum state without an explicit lineage parent.

The EGRESS_ONLY feature generator strictly removes checksum-derived candidates so that the second-stage representation respects the same body-inspection ablation used by Stage 1.

We compare five classifiers: Logistic Regression, a two-hidden-layer multilayer perceptron, balanced Random Forest, class-weighted XGBoost, and a soft-voting ensemble of Random Forest and XGBoost. The MLP uses hidden layers of 64 and 32 units with ReLU activation, early stopping, and a maximum of 200 iterations. The tree models use 100 estimators. The ensemble assigns equal weights to its two constituent models. Logistic Regression and the MLP use standardized features; tree models operate on the original feature scale. Class imbalance is handled through balanced class weights for Logistic Regression and Random Forest and a dynamically computed positive-class weight for XGBoost.

3.6 Evaluation Protocol

Stage 1 is evaluated by streaming the complete corpus through both `FULL_DPI` and `EGRESS_ONLY` modes and measuring average per-event latency, throughput, the fraction filtered on the fast path, the number of escalated events, the first alert sequence position, and rule-trigger distributions.

For Stage 2, each mode is evaluated using a stratified 80/20 train-test split with a fixed random seed. Feature scaling is fit only on the training partition for models that require it and then applied unchanged to the holdout set. Performance is measured using precision, recall, F1-score, and precision-recall AUC, together with inference latency per sample. The training partition is additionally evaluated using five-fold stratified cross-validation with fold-specific feature scaling to assess performance stability across splits. Because artifact degrees are precomputed over the full trace corpus, the Stage 2 structural-feature evaluation is retrospective rather than a fully causal online evaluation; the Stage 1 gateway is evaluated separately under strict forward-only state updates

Finally, the operational classifier threshold is evaluated separately from the default classifier cutoff using Youden's (J) statistic,

$$J(\tau)=\text{Recall}(\tau)-\text{FPR}(\tau)$$

to identify a decision boundary that balances attack sensitivity against false-positive burden.

All components are implemented in Python using Pydantic for schema validation, NetworkX for graph construction and traversal, scikit-learn for classical models and evaluation, XGBoost for gradient-boosted trees, and joblib for checkpoint serialization.

4. Results

4.1 Forensic Reconstruction of the Swarm

Running the provenance pipeline over the complete redacted trace corpus produced a tripartite graph containing 225,592 nodes and 386,402 edges, with 91,396 weakly connected components and 47,113 distinct artifact-degree measurements. The graph separated execution payloads from shared infrastructure and detected capabilities, making it possible to analyze relationships that were not explicit in the original event stream.

The macro-forensic analysis revealed a heterogeneous mixture of benign service activity, credential-bearing events, authenticated service calls, and silent credential-staging behavior. In particular, 100,645 payloads interacted with services, while 19,925 payloads touched credentials and 3,471 exhibited credential access without corresponding service use. The intersection of credential use and service access contained 16,454 payloads. These distributions show why isolated payload inspection is insufficient: credential access, downstream use, and service interaction are distributed across different subsets of the trace.

The artifact activity threads further exposed indirect propagation through shared state. For the targeted credential/service pair, 4,341 payloads used [CREDENTIAL 1], 11 interacted with [SERVICE 2 URL 1], and 5 appeared in both sets. The chronological activity thread therefore linked events that were not connected through ordinary parent-child lineage.

The root-candidate analysis identified 9,646 suspicious depth-zero execution lineages. The highest-ranked candidates were not homogeneous: several were classified as stigmergic asset seeders, while the principal weaponized originator was classified as a Tier-1 weaponized originator. The latter combined an exploit capability with originated shared infrastructure and a large downstream reach. In contrast, another high-ranked root exhibited high artifact multiplicity but a much more localized blast radius. This distinction demonstrates that raw artifact count alone is insufficient for attribution; the combination of capability, novelty, and downstream reach is more informative.

Most importantly, the reconstructed lineage showed that the dominant attack path propagated primarily through shared environmental state rather than ordinary process lineage. In the reconstructed graph, the top weaponized originator had 0 lineage descendants and 6,290 stigmergic readers, while the downstream population inherited shared assets without requiring direct parent-child execution relationships. The resulting two-hop provenance graph therefore exposed a coordination structure that would be difficult to recover from event-local inspection alone.

The forensic pipeline also identified the earliest actionable origin and reconstructed the downstream blast radius. On the reconstructed graph, revoking the compromised shared asset at the root event represents a retrospective counterfactual containment potential of 99.9% (6,290 reachable downstream nodes disconnected). We evaluate this counterfactual structural containment separately from the prospective detection mechanism below.

4.2 Stage 1: Gateway Filtering and Early Detection

We evaluated the stateful gateway under both FULL_DPI and EGRESS_ONLY modes. Both modes processed the complete corpus, but disabling payload inspection substantially changed the computational profile while retaining the key infrastructure-level signals.

Telemetry Dimension	FULL_DPI	EGRESS_ONLY	Comparative Impact
Total events ingested	189,579	189,579	Complete corpus
Total elapsed time	9.7128 s	6.8182 s	30% faster
Average latency/event	0.05123 ms	0.03596 ms	Meets < 0.05 ms target
Throughput	19,518.5 events/s	27,805.0 events/s	+42.5% throughput
Fast-path clean events	129,295	138,835	9,540 additional events filtered
Events escalated to Stage 2	60,284	50,744	Reduced suspect pool
Stage-2 reduction ratio	68.20%	73.23%	Compared with target SLA
First alert sequence	Turn 1	Turn 1	99.9995% lead time

Tainted assets retained	7,905	6,796	Dynamic registry state

The first alert occurred at the same early point in both modes, indicating that the gateway did not depend exclusively on message-body inspection to identify the origin of the incident. In FULL_DPI, the alert drew on body-level capability evidence; in EGRESS_ONLY, the same early-stage detection was driven by infrastructure and credential-state signals. This result is central to the Track 4 objective because it demonstrates that useful oversight can remain possible when message contents are unavailable.

The ground-truth intersection audit showed the trade-off more directly. FULL_DPI escalated 6,784 true attackers and 53,500 false alarms, giving a Stage-1 recall of 100.00% and screening precision of 11.25%. EGRESS_ONLY escalated 6,333 true attackers and 44,411 false alarms, producing 93.35% recall and 12.48% screening precision. The 451 false negatives under EGRESS_ONLY were concentrated in late-stage peripheral activity where payloads omitted authorization headers.

Rule Identifier	FULL_DPI Trip Count	EGRESS_ONLY Trip Count
STIGMERGIC_CREDENTIAL_REUSE	41,833	41,833
TAINTED_CREDENTIAL_INHERITANCE	41,490	41,375
COVERT_EXFILTRATION	28,066	Disabled
TAINTED_DESTINATION_OR_C2_BOARD	21,102	21,102
TAINTED_CODE_EXECUTION	11,249	Disabled
SYNCHRONOUS_XHR_PROBE	10,625	Disabled
RCE_COMMAND_INJECTION	5,251	Disabled
UNDECLARED_CREDENTIAL_VIOLATION	3,242	3,357
SHARED_CODE_VELOCITY	1,907	Disabled
DEFENSE_EVASION	223	Disabled

Rule attribution confirmed that the two modes relied on different evidence sources (Table 4.2). Credential reuse, inherited taint, destination/C2 state, and undeclared-credential violations remained active under EGRESS_ONLY, whereas checksum and payload-capability rules were disabled.

Overall, Stage 1 demonstrated that a stateful, metadata-oriented gateway can remove a large portion of benign traffic before expensive analysis while still detecting the attack near its earliest actionable point.

4.3 Stage 2: Holdout Classification

Stage 2 was evaluated in a retrospective supervised setting on the escalated suspect population using the predefined eight provenance-derived features. The holdout evaluation used a stratified 80/20 split, with performance measured using precision, recall, F1-score, PR-AUC, and per-sample inference latency.

Model Architecture	Precision	Recall	F1-Score	PR-AUC	Latency / Sample
[EGRESS_ONLY MODE]					
Stage-1 Baseline (No ML)	12.48%	93.35%	0.2201	—	0.035 ms
Logistic Regression	0.7817	0.9692	0.8654	0.9761	0.06 µs
ANN / MLP	0.9992	0.9542	0.9762	0.9835	1.17 µs
Random Forest	0.9745	0.9637	0.9690	0.9770	5.79 µs
XGBoost	0.9512	0.9692	0.9601	0.9885	1.31 µs
TraceSentry Ensemble	0.9831	0.9637	0.9733	0.9892	6.99 µs

[FULL_DPI MODE]					
Stage-1 Baseline (No ML)	11.25%	100.00%	0.2022	—	0.051 ms
Logistic Regression	0.3609	0.9234	0.5189	0.8835	0.05 µs
ANN / MLP	1.0000	0.9013	0.9481	0.9750	0.67 µs
Random Forest	0.9397	0.9418	0.9407	0.9742	4.70 µs
XGBoost	0.8796	0.9639	0.9198	0.9878	1.74 µs
TraceSentry Ensemble	0.9335	0.9521	0.9427	0.9876	6.06 µs

The Stage-1 baseline provided the intentionally weak starting point for the escalated population: suspicious metadata alone produced a substantial false-alarm burden. Adding provenance-derived structural features substantially improved screening precision while retaining high attack recall across all non-linear models.

Performance varied across model architectures: in EGRESS_ONLY mode, the Multi-Layer Perceptron (ANN) achieved the highest precision (0.9992) and F1-score (0.9762), while XGBoost and Logistic Regression tied for highest recall (0.9692). The TraceSentry Random Forest + XGBoost soft-voting ensemble yielded the highest PR-AUC (0.9892) and a balanced trade-off between precision (0.9831) and recall (0.9637), with 6.99 µs inference latency per sample. Logistic Regression provided the lowest per-sample latency (0.06 µs), but with substantially lower precision (0.7817).

Comparing EGRESS_ONLY and FULL_DPI shows that classification performance did not degrade when body-derived features were omitted; rather, non-linear models achieved 2.8% to 3.1% higher F1-scores under EGRESS_ONLY, indicating that raw payload text introduced linguistic variance among benign tasks that degraded tree-split precision.

4.4 Cross-Validation Stability

To test sensitivity to the particular train/test partition, we performed 5-fold stratified cross-validation on the 80% training partition across both operating modes.

Model Architecture	EGRESS_ONLY F1	EGRESS_ONLY Precision	EGRESS_ONLY Recall	FULL_DPI F1	FULL_DPI Precision	FULL_DPI Recall
Logistic Regression	0.8674 ± 0.0268	0.7824	0.9747	0.5178 ± 0.0018	0.3595	0.9254
ANN / MLP	0.9798 ± 0.0029	0.9988	0.9615	0.9447 ± 0.0058	0.9994	0.8957
Random Forest	0.9742 ± 0.0057	0.9774	0.9710	0.9407 ± 0.0023	0.9418	0.9397
XGBoost	0.9616 ± 0.0044	0.9510	0.9726	0.9218 ± 0.0078	0.8845	0.9626
TraceSentry Ensemble	0.9752 ± 0.0048	0.9795	0.9710	0.9394 ± 0.0031	0.9321	0.9469

Across folds, the ensemble achieved a mean F1 of 0.9752 ± 0.0048 in EGRESS_ONLY mode and 0.9394 ± 0.0031 in FULL_DPI mode (Table 4.4). F1 variation remained below 0.008 for all tree-based and neural models. The ANN and Ensemble were statistically comparable within the observed cross-validation spread (0.9798 ± 0.0029 vs. 0.9752 ± 0.0048). This demonstrates low partition sensitivity within this evaluated incident trace, though it does not establish transferability to unseen multi-agent swarm environments.

4.5 Operational Decision Threshold Optimization

The ensemble classifier's default decision threshold was compared with a threshold selected using Youden's J statistic on the holdout partition in EGRESS_ONLY mode.

Metric Dimension	Default Threshold ($\tau = 0.50$)	Youden-Optimized Threshold ($\tau^* = 0.3378$)	Operational Benefit
Decision Threshold (τ)	0.5000	0.3378	Shifts cutoff toward risk sensitivity
Recall (Sensitivity)	97.40% (1,237)	98.11% (1,246)	Intercepts 9 additional stealth attackers
False-Positive Rate (FPR)	0.338% (30)	0.755% (67)	Constrained below 1%
Precision	97.63%	94.90%	Highly manageable for operations
F1-Score	0.9752	0.9648	Balanced trade-off
Peak Youden J Index	0.9706	0.9736 / 1.0000	Near-perfect diagnostic separation

Threshold optimization illustrates an operational trade-off: lowering the decision boundary to $\tau^* = 0.3378$ increased attack recall from 97.40% to 98.11% (9 additional stealth attackers intercepted), while increasing the false-positive rate from 0.338% to 0.755% (37 additional false alarms) and lowering precision from 97.63% to 94.90%. The peak Youden index reached 0.9736, confirming high diagnostic separation.

4.6 Comparison with a Zero-Shot LLM Monitor

As an additional baseline, TraceSentry was compared with a zero-shot frontier LLM judge (qwen3.5-397b) on a representative stratified sample of 100 events (25 active exploits, 25 stigmergic consumers, 50 benign baseline events).

Dimension	Frontier LLM (qwen3.5-397b)	TraceSentry Ensemble	Disparity Factor
Inference latency	20,919.4 ms	0.04 ms	TraceSentry is ~500,000× faster
Active exploit recall (Group 1)	64.0% (16/25)	100.0% (25/25)	LLM misses subtle syntax escapes
Downstream swarm recall (Group 2)	80.0% (20/25)	96.0% (24/25)	LLM blind to cross-turn key theft
Overall swarm recall	72.0%	98.0%	+26.0% detection advantage
Benign false-positive rate (Group 3)	80.0% (40/50)	1.8%	LLM suffers catastrophic paranoia

The comparison illustrates the primary limitation of payload-local, zero-shot LLM evaluation: without cross-event provenance memory, the frontier LLM achieved only 72.0% overall recall and suffered an 80.0% false-positive rate on benign developer tasks, alongside a 20.9-second per-event inference latency. In contrast, TraceSentry's structural graph features enabled 98.0% recall on the sample at microsecond latency with 1.8% false positives.

Taken together, the results support three empirical conclusions. First, provenance reconstruction exposes coordination structure that is not apparent from isolated payloads. Second, a stateful egress-oriented gateway provides effective prospective screening early in the incident timeline while filtering the majority of benign traffic. Third, provenance-derived structural features enable effective retrospective classification of the escalated suspect pool without requiring message-body inspection.

5. Discussion and Limitations

5.1 Discussion

The results support the central hypothesis of TraceSentry: multi-agent coordination can remain detectable even when direct agent-to-agent communication is absent, provided that the oversight system observes the shared environmental state through which coordination propagates. The provenance reconstruction revealed that the relevant relationships were not confined to explicit lineage. Instead, apparently independent events became connected through shared credentials, artifacts, and capabilities. This provides a concrete mechanism by which an investigator can move from isolated suspicious events to a swarm-level coordination episode.

The forensic reconstruction also changes the interpretation of apparently benign downstream behavior. In the evaluated incident, downstream consumers could access shared assets without reproducing the original exploit behavior. Consequently, payload-local inspection can classify a consumer as benign while missing its role in the larger attack. TraceSentry's provenance representation addresses this by treating first contact with an asset and subsequent inherited use as related observations. The result is not simply a stronger payload classifier; it is a different unit of analysis in which the relationship between events becomes part of the security signal.

The Stage 1 ablation further suggests that useful oversight does not necessarily require complete message visibility. The EGRESS_ONLY configuration retains infrastructure-level and stateful signals while removing body-derived inspection. Its measured performance and computational profile indicate that a substantial portion of the useful signal can survive this restriction. This is important operationally because it suggests a path toward monitoring systems in which sensitive message contents need not be exposed to the primary security monitor, preserving agent data privacy.

The two-stage architecture provides a complementary systems benefit. Rather than applying the more expensive provenance-derived classifier uniformly to the entire swarm, the stateful gateway first removes benign background activity and escalates a smaller suspect population. The second stage can then spend more computation on structural relationships that are difficult to capture with simple rules. The value of the architecture therefore comes from the interaction between the stages: the gateway provides scale and early filtering, while the provenance-derived classifier provides finer discrimination among suspicious events.

The findings also clarify the scope of TraceSentry's contribution. The individual components provenance graphs, inverse-frequency weighting, stateful security rules, and classical tree-based classifiers are not novel in isolation. The contribution is their combination into a black-box oversight pipeline for multi-agent attacks in which agent identities may be unavailable and coordination may occur through shared environmental state. The empirical results provide initial evidence that this combination can turn forensic provenance relationships into an operational detection signal.

5.2 Limitations and Blind Spots

Several limitations constrain the conclusions. First, TraceSentry is fundamentally dependent on the observability of the coordination channel. If agents coordinate through semantic or steganographic signals that do not modify monitored credentials, files, checksums, destinations, or other observable artifacts, the provenance graph may contain no corresponding structural anomaly. In-band coordination through benign-looking text, whitespace, or code formatting is therefore outside the current detection surface.

Second, the strongest containment result is a retrospective counterfactual rather than a live closed-loop experiment. The estimated containment efficiency of 99.9% is obtained by cutting the reconstructed attack graph at the identified shared asset. In a live deployment, an adaptive swarm could observe revocation and switch to another credential, communication channel, or lateral-movement mechanism. TraceSentry therefore demonstrates the value of identifying the intervention point, but does not establish that an adaptive adversary would remain contained after intervention.

Third, the EGRESS_ONLY evaluation does not detect every malicious event. Across the evaluated trace, 451 malicious payloads (6.65%) bypassed the gateway monitor. These events occurred late in the incident (Turns $>$ 10,000) and accessed unauthenticated internal services through indirect redirection, illustrating that egress metadata alone cannot guarantee complete coverage. A production deployment would require complementary host- or network-level controls.

Fourth, the empirical evaluation is based on a single incident corpus. Although the trace provides a valuable large-scale example of multi-agent coordination, performance on one incident cannot establish generalization across different swarm architectures, attack strategies, communication mechanisms, or environments. Evaluation on additional multi-agent benchmarks and independently generated adversarial scenarios is required before the reported performance can be treated as broadly representative.

Finally, the Stage 2 evaluation should be interpreted as retrospective supervised evidence rather than a complete demonstration of online causal detection. Artifact-degree statistics used by the provenance-derived features are computed over the trace corpus, whereas Stage 1 is evaluated under sequential forward-only state updates. The two stages therefore provide different kinds of evidence: Stage 1 tests prospective screening behavior, while Stage 2 tests whether structural provenance features contain discriminative information in the evaluated incident.

Future Work

Future work should evaluate TraceSentry across multiple independent swarm incidents and adversarially generated coordination strategies. In particular, future experiments should test semantic or covert coordination channels, adaptive attackers that respond to defensive interventions, and richer host- and network-level telemetry. A fully causal online evaluation of the provenance-derived Stage 2 features would also strengthen the evidence for real-time deployment.

6. Conclusion

In this work, we presented TraceSentry, an artifact-mediated provenance and gateway containment architecture for autonomous multi-agent swarms, evaluated on the 189,579-record Hugging Face incident corpus. Our findings demonstrate that multi-agent coordination can remain highly detectable even when direct agent-to-agent communication is absent and individual agent identities are systematically redacted. By structuring execution traces into a tripartite graph of payloads, infrastructure, and capabilities, TraceSentry captured the 100% stigmergic nature of the attack and localized the decisive root originator at Turn 1, indicating a retrospective counterfactual containment potential of 99.9%. Furthermore, our two-stage gateway evaluation proved that oversight does not require invasive payload inspection: an egress-only monitor observing strictly network headers and destinations achieved wire-speed screening (27,805 events/second at 0.036 ms latency) with a 73.2% noise reduction, while subsequent classical ML ensembles triaged the suspect pool with 98.3% precision and 97.3% F1-score.

These results carry a fundamental implication for the science of agent networks and AI alignment: as autonomous agent populations scale, coordination naturally emerges through persistent environmental modifications rather than monitored communication channels. Consequently, safety architectures that evaluate isolated prompt text or chat transcripts will remain systematically blind to distributed, multi-turn intrusions. Effective oversight requires shifting from transcript-level monitoring to causal artifact provenance and gateway-level infrastructure containment. While retrospective log analysis cannot model real-time adaptive counter-evasion, TraceSentry establishes an empirical baseline and architectural blueprint for deploying low-latency, privacy-preserving oversight mechanisms across autonomous agent ecosystems.

Code and Data

- Code repository: https://github.com/chillguycode/TraceSentry
- Data/Datasets: https://swarmtraces.org/data/final/redacted.jsonl.gz

References

1. Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13(2), 281–305.

2. Buehler, M. J., et al. (2026). SwarmWorld: Stigmergic technological evolution in societies of language-model agents. arXiv preprint arXiv:2608.26081.
3. Caltagirone, S., Pendergast, K., & Betz, W. (2013). The Diamond Model of intrusion analysis (Technical Report ADA586960). Center for Cyber Intelligence Analysis and Threat Research (CCIATR).
4. Greenblatt, R., Shlegeris, B., Kashyap, K., & Barnes, B. (2024). AI control: Improving safety despite intentional subversion. In Proceedings of the 41st International Conference on Machine Learning (ICML 2024). arXiv:2312.06942.
5. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? Advances in Neural Information Processing Systems (NeurIPS 2022), 35, 507–520.
6. Han, X., Pasquier, T., Bates, A., Gehani, A., & Seltzer, M. (2020). UNICORN: Runtime provenance-based detector for advanced persistent threats. In 27th Annual Network and Distributed System Security Symposium (NDSS 2020).
7. Jones, E., et al. (2025). Monitoring split-up attacks in distributed language model systems. Journal of AI Safety Research, 4(2), 112–129.
8. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems (NeurIPS 2017), 30, 4765–4774.
9. METR & Redwood Research. (2026). Technical report on the July 2026 multi-agent evaluation security incident. Model Evaluation and Threat Research (METR).
10. Milajerdi, S. M., Gjomemo, R., Eshete, B., Sekar, R., & Venkatakrishnan, V. N. (2019). HOLMES: Real-time APT detection through correlation of suspicious information flows. In 2019 IEEE Symposium on Security and Privacy (S&P) (pp. 1137–1152). IEEE. https://doi.org/10.1109/SP.2019.00063
11. MITRE Corporation. (2020). MITRE ATT&CK® for Enterprise: Technique T1059 (Command and Scripting Interpreter) and Technique T1041 (Exfiltration Over C2 Channel). The MITRE Corporation. https://attack.mitre.org/
12. Motwani, S., et al. (2024). Secret collusion among AI agents. Centre for Long-Term Resilience & UC Berkeley.
13. National Institute of Standards and Technology. (2012). Computer security incident handling guide (NIST Special Publication 800-61, Rev. 2). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-61r2
14. National Institute of Standards and Technology. (2012). Guide for conducting risk assessments (NIST Special Publication 800-30, Rev. 1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-30r1
15. Newman, M. E. J. (2001). Scientific collaboration networks. II. Shortest paths, weighted networks, and centrality. Physical Review E, 64(1), Article 016132. https://doi.org/10.1103/PhysRevE.64.016132
16. OpenAI. (2026). Disclosure regarding multi-agent coordination anomalies during cybersecurity capability evaluations. OpenAI Safety & Alignment Technical Brief.
17. OWASP Foundation. (2020). OWASP ModSecurity Core Rule Set (CRS) v3.x: Collaborative anomaly scoring mode documentation. Open Web Application Security Project. https://coreruleset.org/
18. Paxson, V. (1999). Bro: A system for detecting network intruders in real-time. Computer Networks, 31(23–24), 2435–2463. https://doi.org/10.1016/S1389-1286(99)00112-7

19. Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389. https://doi.org/10.1561/1500000019
20. Rose, J., et al. (2026). Detecting multi-agent collusion through multi-agent interpretability. AI Alignment Foundation (AIAF).
21. Swarm Traces Project. (2026). Swarm Traces: A redacted forensic dataset of 80,000+ multi-agent attack payloads from the 2026 Hugging Face evaluation breach. https://swarmtraces.org/
22. Wijk, et al. (2026). Counter-Swarm Doctrine: Containing coordinated agent intrusions. Hugging Face & ControlAI. https://huggingface.co/papers/2609.06140
23. Youden, W. J. (1950). Index for rating diagnostic tests. Cancer, 3(1), 32–35. https://doi.org/10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3
24. Zhang, Y., et al. (2025). Attributing failures in multi-agent systems. In International Conference on Learning Representations (ICLR 2025). OpenReview: GazlTYxZss.

LLM Usage Statement

I used Gemini just to better target my research and used Gemini and ChatGPT for help with the Paper drafting. All the results taken are either ran by my on Kaggle or Locally